DAYANANDA SAGAR COLLEGE OF ARTS, SCIENCE AND COMMERCE

DEPARTMENT OF COMPUTER APPLICATIONS

(Master of Computer Application)

A Project Report

On

DESIGN AND SIMULATION OF ALU

Under the Guidance

Mrs. Akshatha Deepak

Asst. Professor

Department of Computer Application

Submitted by:

Manan S M – P03CJ25S126039
Pavan Kumar M M – P03CJ25S126059
Puneeth Kumar T A – P03CJ25S126060
Hemanth N – P03CJ25S126024

CONTENT

1. Introduction

2. Objective

3. System Overview

4. Block Diagram Explanation

5. Simulator Details

6. Implementation

7. Logic Design

8. Test Cases and Result

9. Limitation

10. Future Enhancement

11. Conclusion

12. Link of Working Project

DESIGN AND SIMULATION OF ALU

Introduction

The ALU Simulator Using Logic Gates is a digital design project that demonstrates the working of an Arithmetic Logic Unit (ALU) using fundamental digital logic concepts. The system is designed to perform basic arithmetic and logical operations such as addition, subtraction, AND, OR, and NOT on binary inputs.

This project is developed using HTML, CSS, and JavaScript, providing an interactive interface where users can input binary values and select different operations to instantly view the results.

The simulator mimics the behavior of a real ALU by combining multiple logic circuits like adders, multiplexers, and logic gates. It also helps in understanding how different operations are selected using control signals and how the ALU processes data inside a CPU, making it useful for learning both arithmetic and logical computations in digital electronics.

Objective

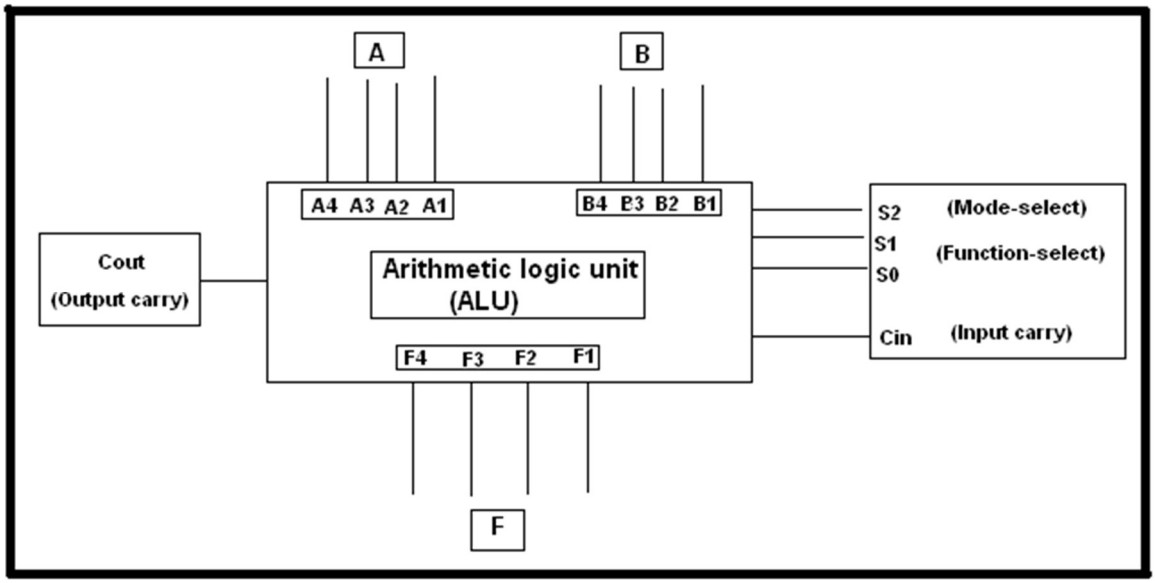
The primary objectives of this project are:

- To perform basic arithmetic operations such as addition and subtraction using binary inputs.
- To execute logical operations like AND, OR, NOT, and XOR.
- To simulate the working of an Arithmetic Logic Unit (ALU) using logic gates.
- To provide an interactive platform for better understanding of digital electronics concepts.
- To demonstrate how control signals select different operations in an ALU.
- To display output results clearly along with relevant flags (such as carry or overflow, if applicable).

System Overview

The system consists of multiple sections:
• Home: Provides an overview of the ALU simulator and its key features.
• Simulator: Allows users to input binary values, select operations, and view results.
• Operations: Displays different arithmetic and logical operations supported by the ALU.
• Circuit: Shows the logic gate implementation and block diagram of the ALU.
• Examples: Includes sample inputs to demonstrate how the ALU works.
• Conclusion: Summarizes the concepts learned and the working of the ALU.

Block Diagram Explanation



This circuit represents a **4-bit ALU (Arithmetic Logic Unit)** constructed using basic digital components. It is designed to perform both arithmetic and logical operations on binary inputs.

Main Components in Diagram

1. Inputs

- A3, A2, A1, A0 → First binary number (A)
- B3, B2, B1, B0 → Second binary number (B)
- Control Signals → Used to select the required operation
- Each pair (Ai, Bi) is processed by the internal ALU circuits

2. Logic Gates (Core Role)

The circuit uses fundamental gates such as AND, OR, NOT, and XOR to perform logical operations:

- AND → Produces output when both inputs are 1
- OR → Produces output when at least one input is 1
- XOR → Used for addition and comparison operations
- NOT → Inverts the input signal

3. Arithmetic Unit (Adder/Subtractor)

The arithmetic section is built using full adders:

- Performs Addition (A + B)
- Performs Subtraction (A − B) using 2's complement method
- Uses XOR gates and carry input to switch between operations

4. Control Unit (Operation Selection)

- Control signals determine which operation is executed
- A multiplexer-like selection mechanism chooses between arithmetic and logic outputs
- Enables switching between multiple ALU functions

5. Carry Chain (Ripple Carry Mechanism)

- Carry propagates through each stage of the full adders
- Moves from least significant bit (LSB) to most significant bit (MSB)
- Helps in performing multi-bit arithmetic operations

6. Output Section

- Result is produced as output bits (S0, S1, S2, S3)
- Carry-out and/or overflow flags may also be generated
- Final output depends on the selected operation

This design helps in understanding how an ALU integrates arithmetic and logical circuits, and how control signals coordinate different operations inside a processor.

Simulator Details

The simulator is developed using HTML, CSS, and JavaScript.

- HTML: Structure of the web page.
- CSS: Styling and layout.
- JavaScript: Logic implementation.

Features:
- Interactive bit input (toggle between 0 and 1)
- Addition and subtraction modes
- Step-by-step calculation display
- Carry and overflow detection

Implementation

The implementation explains the logic used to design a **4-bit ALU (Arithmetic Logic Unit)** that performs both arithmetic and logical operations using digital circuits.

A. Full Adder Logic

The fundamental building block of the ALU is the **Full Adder**, which adds three input bits (A, B, Cin).

Full Adder Truth Table:

A	B	Cin	Sum (S)	Carry Out (Cout)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Equations:

- Sum (S): $S = A \oplus B \oplus Cin$
- Carry Out (Cout): $Cout = (A \cdot B) + (Cin \cdot (A \oplus B))$

B. Subtraction Logic (Using 2's Complement)

Subtraction is performed by converting it into addition using 2's complement.

Working Principle:

- Input B is conditionally inverted using XOR logic
- $B' = B \oplus Mode$
- If Mode = 0 → Normal addition
- If Mode = 1 → B is inverted (1's complement)

Initial Carry Input:

- Cin = Mode
- This adds 1 to complete the 2's complement

So:

- Mode = 0 → Addition (A + B)
- Mode = 1 → Subtraction (A − B)

C. Logic Operations
The ALU also performs basic logical operations:
• AND: Output is 1 only if both inputs are 1
• OR: Output is 1 if at least one input is 1
• XOR: Output is 1 if inputs are different
• NOT: Inverts the input

D. Signed and Unsigned Representation
• **Unsigned Numbers:** Range from 0 to 15 (for 4-bit)
• **Signed Numbers (2's Complement):** Range from −8 to +7
Conversion Rule:
• If MSB (S3) = 1 → Number is negative
• Signed value = Unsigned value − 16

E. Overflow Detection
Overflow occurs only in signed arithmetic when the result exceeds the valid range.
Logic:
• Overflow = C3 ⊕ C4
• Where:
  – C3 = Carry into MSB
  – C4 = Carry out of MSB

Logic Design

The implementation includes the following logic:

1. **Binary Operations:**

- Performs arithmetic operations like addition and subtraction on binary inputs.

2. **Logical Operations:**

- Executes bitwise operations such as AND, OR, XOR, and NOT.

3. **Control Signal Selection:**

- Uses control inputs to select the required operation within the ALU.

4. **Arithmetic Processing:**

- Utilizes full adders and logic gates to carry out computations step by step.

5. **Carry Handling:**

- Propagates carry across multiple bits using ripple carry mechanism.

6. **Overflow Detection:**

- Detects overflow when the result exceeds the valid range in signed arithmetic.

Test Cases & Results

Test Case 1:
A = 0101, B = 0011, Operation = + → Result = 1000, Overflow = No
Test Case 2:
A = 1010, B = 0101, Operation = + → Result = 1111, Overflow = No
Test Case 3:
A = 0110, B = 0010, Operation = − → Result = 0100, Overflow = No
Test Case 4:
A = 0111, B = 0001, Operation = + → Result = 1000, Overflow = Yes
Test Case 5:
A = 1000, B = 1000, Operation = + → Result = 0000, Overflow = Yes

Limitations

• Limited to 4-bit binary operations
• Supports only basic arithmetic and logic functions
• No support for decimal or floating-point calculations
• Accuracy depends on correct binary input selection
• Overflow handling is limited to small bit-width operations

Future Enhancements

• Extend to support higher bit-width operations (8-bit, 16-bit, etc.)
• Include decimal and signed number output display
• Enhance user interface with animations and better visualization
• Add advanced operations like multiplication and division
• Integrate step-by-step execution for better understanding

Conclusion

This project demonstrates how an Arithmetic Logic Unit (ALU) performs both arithmetic and logical operations using fundamental digital logic circuits. It helps in understanding how core components of a processor work together to process binary data.

Key Findings:

- Interactive input selection for binary values
- Real-time execution of operations
- Support for arithmetic and logical functions
- Control-based operation selection
- Clear display of results and output states

Link of the Project

https://github.com/hemanthgowdahari-dev/ALU-Simulator/tree/main